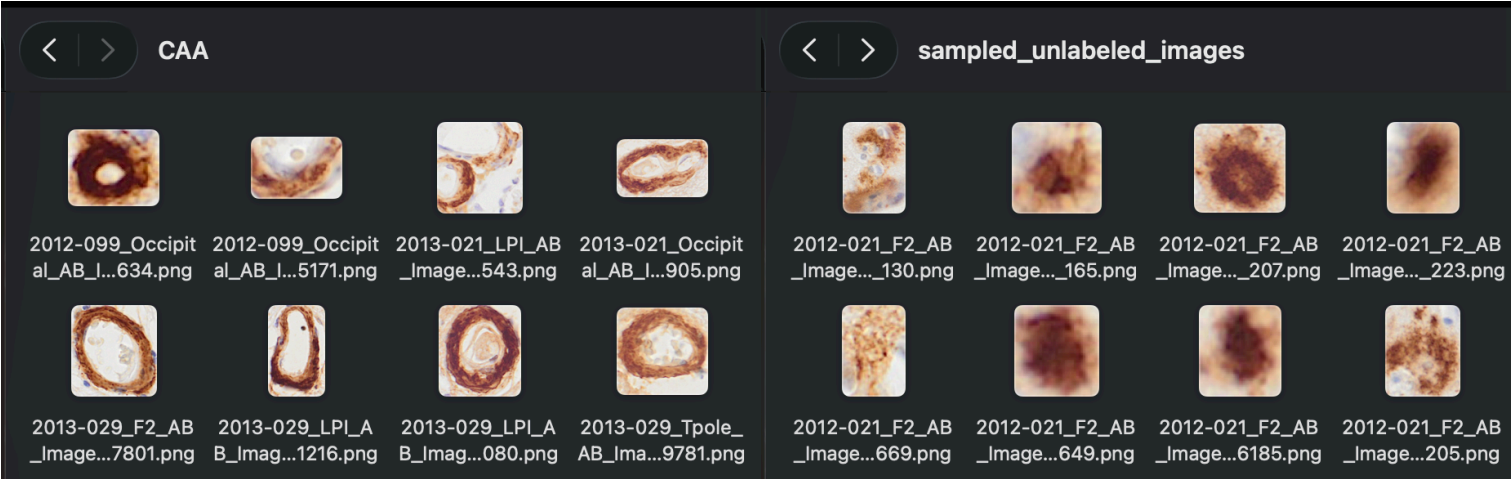
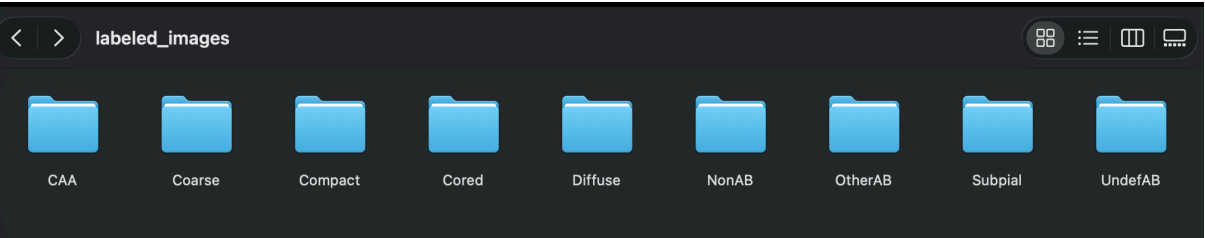


Honours Project Report

Already Implemented:

1- Data Preparation:

Segmented plaque images were extracted from large HDF5 files and organized into folders by label. For each plaque, additional morphological features (roundness and area) were extracted from the HDF5 metadata and stored in a data table alongside the corresponding image filename and index. The data table contains columns for Image (source HDF5 filename), Index (plaque identifier within the file), Roundness, Area, and Label (for labeled samples). Labeled images are stored in subdirectories named by their class label (e.g., Diffuse, Compact, Cored, Subpial, CAA), while unlabeled images are stored in a separate folder. Each segmented plaque is saved as a PNG file with the naming convention “{image_name}_index_{index}.png”. All images were downsampled to 224×224 pixels to ensure uniform dimensions for training. Given the large number of unlabeled plaques (approximately 4 million), a random sample of 10,000 unlabeled plaques was selected to create a manageable dataset for training and evaluation.



data_table_sampled_10k

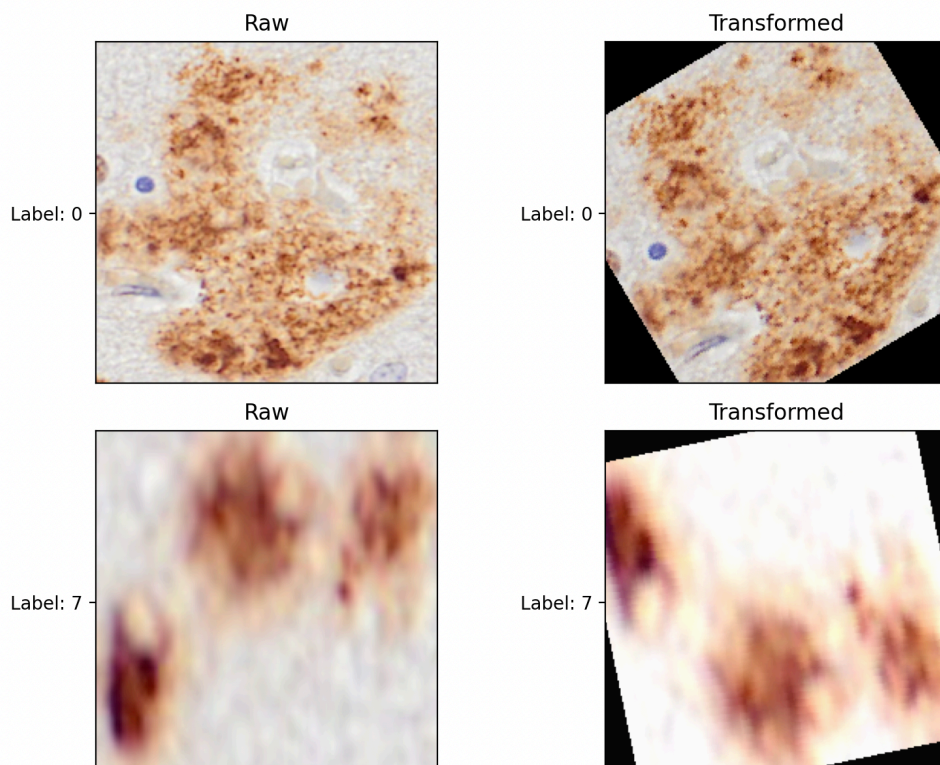
Image	Index	Roundness	Area	Label
Image_2014-010_Occipital_AB.hdf5	10985	0.3599173420565950	880.5664040000000	Diffuse
Image_2016-008_F2_BA4.hdf5	2444	0.5115943894106370	1326.6679960000000	Diffuse
Image_2019-025_LPI_AB.hdf5	10349	0.3008987961916100	1029.4421120000000	Diffuse
2018-014_F2_AB_Image.hdf5	9700	0.6879559486933610	347.3766520000000	Diffuse
Image_2017-027_F2_BA4.hdf5	9487	0.6519894635419590	256.8349960000000	Diffuse
Image_2014-079_F2_AB.hdf5	10812	0.37812738499065800	205.8583920000000	Diffuse
Image_2013-052_LPI_AB.hdf5	9203	0.4314351613764410	164.2662880000000	Diffuse
Image_2014-010_Occipital_AB.hdf5	11223	0.042348679764331400	10657.338504000000	Subpial
2015-067_F2_AB_Image.hdf5	4004	0.675487478161061	207.0596080000000	Compact
Image_2021-001_tamnidA_AB4.hdf5	204	0.5810831902771800	2132.9091600000000	Cored

2- Data Preprocessing:

The next step was designing a data preprocessing pipeline. For this, **PlaqueDataset** module was implemented to handle image loading, normalization, transformation, and augmentation. The class accepts parameters for normalization statistics (mean and standard deviation), transformation pipelines.

The **PlaqueDataset** returns multiple outputs: the raw normalized image tensor, a stacked tensor of all transformed/augmented views, extra morphological features (if enabled), and the label. When multiple transforms are provided as a list, all transformed views are stacked along a new dimension, allowing simultaneous access to different augmentations of the same image. This design is useful for semi-supervised and self-supervised learning, where pairs of augmentations (e.g., weak and strong) are needed for consistency regularization.

To support supervised learning, an augmented variant called **PlaqueDatasetAugmented** module was implemented. This variant expands the dataset by treating each augmentation as a separate sample. Internally, it creates multiple **PlaqueDataset** instances one for each augmentation plus one for the raw image effectively multiplying the dataset size by (number_of_augmentations+1). This allows augmentations to serve as additional training samples that can be shuffled independently by the data loader, which is beneficial for supervised approaches. In contrast, the standard **PlaqueDataset** preserves the relationship between different views of the same image, which is essential for consistency-based semi-supervised methods. Both dataset classes support optional data preloading for faster training, on-the-fly transformation application, and configurable image downscaling methods (bilinear or nearest neighbor). The specific transformations used vary by architecture and training approach, and will be detailed in the respective sections.



3- Configuration Manager System

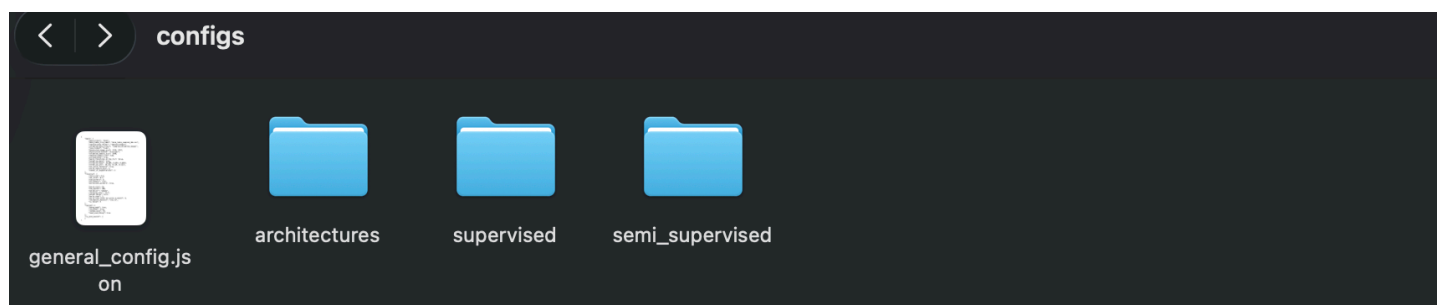
A configuration management system was implemented to centralize experiment settings and simplify parameter management. The **Config** class loads and merges JSON files from a structured directory hierarchy, providing a hierarchical, object-like interface.

The configuration directory is organized hierarchically. At the root level, `general_config.json` contains shared settings for data preprocessing, training hyperparameters, and system configuration. Subdirectories organize mode-specific and model-specific configurations: **supervised** folder contains supervised learning configurations, **semi_supervised** folder contains semi-supervised method configurations (e.g., Mean Teacher, FixMatch, Pi-Model), and **architecture** folder contains feature extractor and classifier configurations. Each model has its own configuration file, allowing independent parameter management.

The **Config** class recursively loads all JSON files in this structure and converts nested dictionaries into nested **Config** objects. This enables dot-notation access like `config.general_config.training.batch_size` instead of dictionary indexing, making the configuration more readable and easier to use throughout the codebase.

When loading configurations, the system automatically performs several post-processing steps. It loads label mappings from a CSV file and adds them to the config object. It generates a unique run ID (either from the SLURM job ID if available, or from the current timestamp). It automatically detects and sets the device to "cuda" if available, otherwise "cpu". Finally, it filters out irrelevant configuration sections based on the training mode (supervised, semi_supervised, or self_supervised), ensuring that only the relevant configurations are loaded for each experiment type.

This configuration system enables easy experimentation by allowing researchers to modify JSON files without changing code, and it ensures reproducibility by saving the complete configuration with each experiment run.



An example of general_config.json

configs > {} general_config.json > ...

```
1  {
2      "data": {
3          "data_folder": "data",
4          "data_table_file_name": "data_table_sampled_10k.csv",
5          "labeled_data_folder": "labeled_images",
6          "unlabeled_data_folder": "sampled_unlabeled_images",
7          "runs_folder": "runs",
8          "downscaled_image_size": [224, 224],
9          "downscaling_method": "bilinear",
10         "unlabeled_sample_size": 2000,
11         "labeled_sample_size": 1e9,
12         "preload_data": true,
13         "apply_transforms_on_the_fly": false,
14         "normalize_data": true,
15         "normalize_mean": [0.485, 0.456, 0.406],
16         "normalize_std": [0.229, 0.224, 0.225],
17         "use_extra_features": true,
18         "extra_feature_dim": 2,
19         "number_of_augmentations": 3
20     },
21     "training": {
22         "test_size": 0.2,
23         "val_size": 0.1,
24         "num_workers": 4,
25         "pin_memory": true,
26         "persistent_workers": true,
27
28         "batch_size": 64,
29         "num_epochs": 500,
30         "optimizer": "adamw",
31         "learning_rate": 1e-4,
32         "weight_decay": 1e-5,
33         "early_stop": 2,
34         "early_stop_check_val_every_n_epoch": 5,
35         "checkpoint_monitor": "val_f1",
36         "cv_folds": 5
37     },
38     "system": {
39         "debug_mode": true,
40         "log_mode": true,
41         "random_seed": 44,
42         "seed_everything": true
43     },
44     "cv_grid_search": {
45     }
46 }
```

And then further on in **main.py**, which is the entry point of the program, the **config_dir** is given as a parameter alongside the **train_mode** and the **run_mode**, and then a **Runner** class instance is created, which is responsible for connecting all the data preprocessing logic to the actual training logic and logging and saving the results. We will not go into the details of them in this report.

```
src > main.py > ...
1  import argparse
2
3  from utils.logging_utils import print_log
4  from models.config import Config
5  from models.base_runner import BaseRunner
6
7  if __name__ == "__main__":
8      # Parse arguments with config files
9      parser = argparse.ArgumentParser(description="Plaque Analysis with Config Files")
10
11     parser.add_argument(
12         "--config_dir",
13         type=str,
14         default="configs",
15         help="Directory containing config files",
16     )
17     parser.add_argument(
18         "--train_mode",
19         type=str,
20         default="supervised",
21         choices=["supervised", "semi_supervised", "self_supervised"],
22         help="Training mode to use",
23     )
24     parser.add_argument(
25         "--run_mode",
26         type=str,
27         default="single",
28         choices=["single", "cross_validate", "optimize_hyperparameters"],
29         help="Run mode to use",
30     )
31     # Parse arguments
32     args = parser.parse_args()
33
34     # Load and merge configurations
35     config = Config.load_config(args.config_dir, args.train_mode)
36
37     print_log(
38         "Config: " + str(config),
39         log_mode=config.general_config.system.log_mode,
40         end="\n\n",
41     )
42     runner = BaseRunner.create_runner(args.train_mode, config)
43
44     if args.run_mode == "single":
45         runner.run_single_experiment()
46     elif args.run_mode == "cross_validate":
47         runner.cross_validate()
48     elif args.run_mode == "optimize_hyperparameters":
49         runner.optimize_hyperparameters()
50     else:
51         raise ValueError(f"Invalid run mode: {args.run_mode}")
```

4- Supervised base model:

A supervised baseline model was implemented to compare semi-supervised and self-supervised approaches. The implementation uses PyTorch Lightning for training, validation, and testing.

The architecture is modular with two components: a feature extractor and a classification head. The feature extractor supports multiple ResNet backbones (ResNet-18, ResNet-34, ResNet-50) using ImageNet pretrained weights. The final classification layer is removed, and the backbone is followed by a linear projection layer with ReLU activation and dropout regularization. The feature extractor can be frozen or fine-tuned and for this part we froze the feature extractor (resnet model) and only trained the linear head.

The classification head can be either a linear layer or a multi-layer perceptron (MLP) with configurable hidden layers and dropout. Optionally, morphological features (roundness and area) can be concatenated with the image features before classification, providing additional information to the model.

The model is implemented as a PyTorch Lightning module (**LightningSupervisedModule**) that encapsulates the feature extractor and classifier, handles training/validation/test steps, and manages metrics tracking. Training uses cross-entropy loss on labeled data only, with standard data augmentation and normalization techniques. This supervised baseline serves as the reference point for evaluating the performance improvements achieved by semi-supervised and self-supervised learning methods. The following section presents the performance results of this baseline model.

Results of 5-fold cross validation:

	precision	recall	f1-score	support
CAA	0.842 ± 0.179	0.915 ± 0.121	0.872 ± 0.063	16.2 ± 0.8
Coarse	0.791 ± 0.1	0.786 ± 0.137	0.785 ± 0.061	21.4 ± 0.98
Compact	0.883 ± 0.209	0.84 ± 0.185	0.854 ± 0.134	12.6 ± 0.98
Cored	0.728 ± 0.083	0.822 ± 0.247	0.77 ± 0.142	15.8 ± 0.8
Diffuse	0.894 ± 0.133	0.987 ± 0.054	0.937 ± 0.08	14.8 ± 0.8
NonAB	0.913 ± 0.221	0.973 ± 0.066	0.938 ± 0.129	14.6 ± 0.98
OtherAB	0.463 ± 0.689	0.195 ± 0.252	0.255 ± 0.308	6.2 ± 0.8
Subpial	0.766 ± 0.306	0.647 ± 0.179	0.698 ± 0.223	8.6 ± 0.98
UndefAB	0.95 ± 0.2	0.8 ± 0.339	0.858 ± 0.233	7.6 ± 0.98
macro avg	0.803 ± 0.152	0.774 ± 0.098	0.774 ± 0.109	117.8 ± 0.8
weighted avg	0.82 ± 0.1	0.822 ± 0.076	0.811 ± 0.082	117.8 ± 0.8

Transformation used to augment new labeled samples to training set:

```
src > supervised_runner.py > SupervisedRunner > _load_data loaders
36 class SupervisedRunner(BaseRunner):
365 def _load_data loaders(
375     train_transforms = trf.Compose(
376         [
377             trf.RandomHorizontalFlip(p=0.5),
378             trf.RandomVerticalFlip(p=0.5),
379             trf.RandomRotation(degrees=(0, 90)),
380             trf.ColorJitter(brightness=0.2, contrast=0.2),
381             trf.ToTensor(),
382         ]
383     )
```

Configuration:

```
{
  supervised: {
    supervised_config: {
      feature_extractor_name: resnet18,
      classifier_name: linear,
    }
  },
  architectures: {
    classifiers_config: {
      linear: {},
    },
    feature_extractors_config: {
      resnet18: {
        output_size: 64,
        pretrained: True,
```

```

        freeze_feature_extractor: True,

        dropout_rate: 0.2,

    },

}

},

general_config: {

    data: {

        unlabeled_sample_size: 2000,

        labeled_sample_size: 1000000000.0,

        preload_data: True,

        apply_transforms_on_the_fly: False,

        normalize_data: True,

        normalize_mean: [0.485, 0.456, 0.406],

        normalize_std: [0.229, 0.224, 0.225],

        use_extra_features: True,

        extra_feature_dim: 2,

        number_of_augmentations: 3

    },

    training: {

        test_size: 0.2,

        val_size: 0.1,

        batch_size: 64,

        num_epochs: 500,

        optimizer: adamw,

        learning_rate: 0.0001,

        weight_decay: 1e-05,

        early_stop: 2,

        early_stop_check_val_every_n_epoch: 5,

        cv_folds: 5

    },

    system: {

        random_seed: 44

    },

},

}

```


5- Semi-supervised models:

5-1 PI model:

The PI Model (Π -Model) was implemented as a semi-supervised learning approach using consistency regularization. The idea is that different augmentations of the same image should produce similar predictions.

The implementation extends the base semi-supervised module and follows a two-stream training process. For labeled data, it computes a standard supervised loss using ground truth labels. For unlabeled data, it applies two augmentations to each image: a weak augmentation (minimal transformations) and a strong augmentation (more aggressive transformations). Both augmented versions are passed through the same model to obtain predictions.

The consistency loss measures the disagreement between the predictions on the weak and strong augmentations, encouraging the model to be invariant augmentation transformations, effectively learning from unlabeled data. The consistency loss can be computed using different metrics, such as mean squared error between the prediction distributions.

The total training loss combines the supervised loss on labeled data and the consistency loss on unlabeled data, weighted by a consistency coefficient. To prevent early instability, this coefficient starts small and gradually increases during training using a ramp-up schedule. This allows the model to first learn from labeled data before incorporating unlabeled data.

This approach leverages unlabeled data by enforcing prediction consistency across augmentations, improving generalization without requiring additional labeled examples.

Algorithm 1 Π -model pseudocode.

Require: x_i = training stimuli

Require: L = set of training input indices with known labels

Require: y_i = labels for labeled inputs $i \in L$

Require: $w(t)$ = unsupervised weight ramp-up function

Require: $f_\theta(x)$ = stochastic neural network with trainable parameters θ

Require: $g(x)$ = stochastic input augmentation function

for t in $[1, num_epochs]$ **do**

for each minibatch B **do**

$z_{i \in B} \leftarrow f_\theta(g(x_{i \in B}))$

$\tilde{z}_{i \in B} \leftarrow f_\theta(g(x_{i \in B}))$

$loss \leftarrow -\frac{1}{|B|} \sum_{i \in (B \cap L)} \log z_i[y_i]$
 $+ w(t) \frac{1}{C|B|} \sum_{i \in B} ||z_i - \tilde{z}_i||^2$

 update θ using, e.g., ADAM

end for

end for

return θ

▷ evaluate network outputs for augmented inputs

▷ again, with different dropout and augmentation

▷ supervised loss component

▷ unsupervised loss component

▷ update network parameters

Results of 5-fold cross validation:

	precision	recall	f1-score	support
CAA	0.874 ± 0.163	0.729 ± 0.146	0.791 ± 0.104	16.2 ± 0.8
Coarse	0.676 ± 0.056	0.813 ± 0.137	0.736 ± 0.054	21.4 ± 0.98
Compact	0.778 ± 0.089	0.732 ± 0.21	0.752 ± 0.131	12.6 ± 0.98
Cored	0.683 ± 0.17	0.76 ± 0.288	0.715 ± 0.195	15.8 ± 0.8
Diffuse	0.787 ± 0.17	0.972 ± 0.068	0.869 ± 0.122	14.8 ± 0.8
NonAB	0.707 ± 0.141	0.987 ± 0.054	0.821 ± 0.089	14.6 ± 0.98
OtherAB	0.4 ± 0.98	0.067 ± 0.164	0.114 ± 0.28	6.2 ± 0.8
Subpial	0.8 ± 0.8	0.275 ± 0.277	0.409 ± 0.41	8.6 ± 0.98
UndefAB	0.819 ± 0.298	0.686 ± 0.102	0.738 ± 0.115	7.6 ± 0.98
macro avg	0.725 ± 0.199	0.669 ± 0.049	0.66 ± 0.082	117.8 ± 0.8
weighted avg	0.738 ± 0.118	0.74 ± 0.044	0.714 ± 0.057	117.8 ± 0.8

```
src > semi_supervised_runner.py > SemiSupervisedRunner > _load_data loaders
37 class SemiSupervisedRunner(BaseRunner):
371 def _load_data loaders(
372     # weak augmentation: minimal changes
391     weak_transforms = trf.Compose(
392         [
393             trf.RandomHorizontalFlip(p=0.5),
394             trf.RandomVerticalFlip(p=0.5),
395             trf.ToTensor(),
396         ]
397     )
398     strong_transforms = trf.Compose(
399         [
400             trf.RandAugment(num_ops=2, magnitude=10),
401             trf.ToTensor(),
402         ]
403     )
```

Note that all the different semi-supervised models introduced later use these transformations.

Configuration:

```
{
    semi_supervised: {
        pi_model_config: {

        },
        semi_supervised_config: {
            model_name: pi_model,
            feature_extractor_name: resnet18,
            classifier_name: linear,
            training: {
                consistency_lambda_max: 0.1,
                consistency_loss_type: mse,
                ramp_up_epochs: 100,
```

```

        ramp_up_function: linear
    }
}
},
architectures: {
    classifiers_config: {
        linear: {},
    },
    feature_extractors_config: {
        resnet18: {
            output_size: 64,
            pretrained: True,
            freeze_feature_extractor: True,
            dropout_rate: 0.2,
        },
    }
},
general_config: {
    data: {
        unlabeled_sample_size: 2000,
        labeled_sample_size: 1000000000.0,
        preload_data: True,
        apply_transforms_on_the_fly: False,
        normalize_data: True,
        normalize_mean: [0.485, 0.456, 0.406],
        normalize_std: [0.229, 0.224, 0.225],
        use_extra_features: True,
        extra_feature_dim: 2,
        number_of_augmentations: 3
    },
    training: {
        test_size: 0.2,
        val_size: 0.1,
        batch_size: 64,
        num_epochs: 500,
        optimizer: adamw,
        learning_rate: 0.0001,
        weight_decay: 1e-05,
    }
}

```

```
        early_stop: 2,

        early_stop_check_val_every_n_epoch: 5,

        cv_folds: 5

    },

    system: {

        random_seed: 44

    },

},

}
```

5-2 FixMatch:

Following the PI model, FixMatch was implemented as a semi-supervised approach that combines pseudo-labeling with consistency regularization. It adds confidence-based filtering to focus on high-confidence unlabeled samples.

The approach uses a two-stage process for unlabeled data. First, weakly augmented images are passed through the model to obtain prediction probabilities. The maximum softmax probability is used as a confidence score. If this score exceeds a predefined threshold, the predicted class becomes a pseudo-label for that sample. This threshold acts as a quality filter, ensuring only confident predictions are used.

In the second stage, strongly augmented versions of the same images are passed through the model. The consistency loss is computed only for samples that passed the confidence threshold, using cross-entropy between the strong augmentation predictions and the pseudo-labels derived from the weak augmentations. Samples below the threshold are ignored, preventing the model from learning from uncertain predictions.

The total training loss combines the supervised loss on labeled data with the consistency loss on high-confidence unlabeled samples, weighted by a consistency coefficient that ramps up during training. This selective use of unlabeled data helps the model learn from reliable pseudo-labels while avoiding noise from uncertain predictions, improving generalization with limited labeled data.

Algorithm 1 FixMatch algorithm.

- 1: **Input:** Labeled batch $\mathcal{X} = \{(x_b, p_b) : b \in (1, \dots, B)\}$, unlabeled batch $\mathcal{U} = \{u_b : b \in (1, \dots, \mu B)\}$, confidence threshold τ , unlabeled data ratio μ , unlabeled loss weight λ_u .
 - 2: $\ell_s = \frac{1}{B} \sum_{b=1}^B H(p_b, \alpha(x_b))$ {Cross-entropy loss for labeled data}
 - 3: **for** $b = 1$ **to** μB **do**
 - 4: $q_b = p_m(y \mid \alpha(u_b); \theta)$ {Compute prediction after applying weak data augmentation of u_b }
 - 5: **end for**
 - 6: $\ell_u = \frac{1}{\mu B} \sum_{b=1}^{\mu B} \mathbb{1}\{\max(q_b) > \tau\} H(\arg \max(q_b), p_m(y \mid \mathcal{A}(u_b)))$ {Cross-entropy loss with pseudo-label and confidence for unlabeled data}
 - 7: **return** $\ell_s + \lambda_u \ell_u$
-

Results of 5-fold cross validation:

	precision	recall	f1-score	support
CAA	0.89 ± 0.095	0.778 ± 0.057	0.829 ± 0.046	16.2 ± 0.8
Coarse	0.691 ± 0.191	0.822 ± 0.245	0.742 ± 0.138	21.4 ± 0.98
Compact	0.821 ± 0.089	0.639 ± 0.362	0.705 ± 0.185	12.6 ± 0.98
Cored	0.638 ± 0.105	0.76 ± 0.356	0.685 ± 0.188	15.8 ± 0.8
Diffuse	0.847 ± 0.2	0.972 ± 0.068	0.903 ± 0.125	14.8 ± 0.8
NonAB	0.768 ± 0.083	0.987 ± 0.054	0.863 ± 0.042	14.6 ± 0.98
OtherAB	0.38 ± 0.742	0.167 ± 0.298	0.21 ± 0.351	6.2 ± 0.8
Subpial	0.767 ± 0.777	0.367 ± 0.395	0.49 ± 0.505	8.6 ± 0.98
UndefAB	0.88 ± 0.244	0.789 ± 0.129	0.826 ± 0.123	7.6 ± 0.98
macro avg	0.742 ± 0.183	0.698 ± 0.072	0.695 ± 0.107	117.8 ± 0.8
weighted avg	0.757 ± 0.114	0.757 ± 0.041	0.737 ± 0.066	117.8 ± 0.8

Configuration:

```
{
  semi_supervised: {
    fixmatch_config: {
      pseudo_label_confidence_threshold: 0.95
    },
    semi_supervised_config: {
      model_name: fixmatch,
      feature_extractor_name: resnet18,
      classifier_name: linear,
      training: {
        consistency_lambda_max: 0.1,
        consistency_loss_type: cross_entropy,
        ramp_up_epochs: 100,
        ramp_up_function: linear
      }
    }
  },
  architectures: {
    classifiers_config: {
      linear: {},
    },
    feature_extractors_config: {
      resnet18: {
        output_size: 64,
        pretrained: True,
```

```

        freeze_feature_extractor: True,

        dropout_rate: 0.2,

    },

}

},

general_config: {

    data: {

        unlabeled_sample_size: 2000,

        labeled_sample_size: 1000000000.0,

        preload_data: True,

        apply_transforms_on_the_fly: False,

        normalize_data: True,

        normalize_mean: [0.485, 0.456, 0.406],

        normalize_std: [0.229, 0.224, 0.225],

        use_extra_features: True,

        extra_feature_dim: 2,

        number_of_augmentations: 3

    },

    training: {

        test_size: 0.2,

        val_size: 0.1,

        batch_size: 64,

        num_epochs: 500,

        optimizer: adamw,

        learning_rate: 0.0001,

        weight_decay: 1e-05,

        early_stop: 2,

        early_stop_check_val_every_n_epoch: 5,

        cv_folds: 5

    },

    system: {

        random_seed: 44

    },

},

}

```

5-3 MeanTeacher:

After FixMatch, the Mean Teacher method was implemented as another semi-supervised approach, this time based on a teacher–student framework. The core idea is to use a slowly updated teacher model to provide stable targets for consistency regularization on unlabeled data.

In this setup, the student model is the main network that is trained with gradient descent, while the teacher model is a separate copy whose weights are updated as an exponential moving average (EMA) of the student's weights. At each training step, the student sees one augmented version of an unlabeled image (typically the stronger augmentation) and produces a prediction. The teacher sees another augmented version of the same image (typically the weaker augmentation) and produces its own prediction, but without gradients.

A consistency loss is then computed between the student and teacher predictions on these two views of the same unlabeled sample. This encourages the student to match the teacher's more stable predictions, even under stronger augmentations. The supervised loss on labeled data and the consistency loss on unlabeled data are combined, with the consistency term weighted by a coefficient that ramps up over time.

During training, the teacher's weights are continuously updated via EMA after each batch, gradually smoothing out the student's fluctuations. At the end of training, the teacher model is used for validation and inference, leveraging its more stable representation learned from both labeled and unlabeled data.

Algorithm 1: Mean Teacher Algorithm

Data: train set $(\mathcal{X}, \mathcal{Y})$, Unlabel data $(\mathcal{Z})$
Hyper parameters: $r, \alpha, p1, p2, \rho1, \rho2, epochs$;
Create Model : $student(\theta), teacher(\theta')$;
Train *teacher* for 1 epoch;
while *epochs* **do**
 while *steps* **do**
 1: Insert noise with probability as per strategies ($p1, p2, \rho1, \rho2$) in $\mathcal{X}$ i.e. $\mathcal{X}_\eta$;
 2: $student(\mathcal{X}_\eta) = \mathcal{Y}_\eta$;
 3: Classification cost ($C(\theta)$) = Binary Cross Entropy($\mathcal{Y}, \mathcal{Y}_\eta$);
 4: Again create different noise data as mentioned in step 1 i.e. $\mathcal{X}_{\eta'}$;
 5: $teacher(\mathcal{X}_{\eta'}) = \mathcal{Y}_{\eta'}$;
 6: Calculate Consistency cost $J(\theta)$ = Mean Squared Error($\mathcal{Y}_\eta, \mathcal{Y}_{\eta'}$);
 7: Calculate Overall cost
 $O(\theta) = \lambda C(\theta) + (1 - \lambda) J(\theta)$;
 8: Calculate *gradients*, $O(\theta)$ w.r.t θ ;
 9: Apply *gradients* to θ ;
 10: Update Exponential Moving average of θ to θ' i.e. $\theta'_t = \alpha \theta'_{t-1} + (1 - \alpha) \theta_t$;
 end
end

Results of 5-fold cross validation:

	precision	recall	f1-score	support
CAA	0.872 ± 0.149	0.877 ± 0.072	0.873 ± 0.088	16.2 ± 0.8
Coarse	0.696 ± 0.144	0.879 ± 0.142	0.772 ± 0.077	21.4 ± 0.98
Compact	0.842 ± 0.206	0.718 ± 0.41	0.748 ± 0.235	12.6 ± 0.98
Cored	0.728 ± 0.102	0.772 ± 0.343	0.744 ± 0.211	15.8 ± 0.8
Diffuse	0.861 ± 0.208	0.987 ± 0.054	0.916 ± 0.122	14.8 ± 0.8
NonAB	0.834 ± 0.155	0.987 ± 0.054	0.902 ± 0.099	14.6 ± 0.98
OtherAB	0.433 ± 0.778	0.133 ± 0.249	0.196 ± 0.346	6.2 ± 0.8
Subpial	0.827 ± 0.517	0.369 ± 0.267	0.505 ± 0.335	8.6 ± 0.98
UndefAB	0.872 ± 0.215	0.743 ± 0.275	0.788 ± 0.116	7.6 ± 0.98
macro avg	0.774 ± 0.168	0.718 ± 0.084	0.716 ± 0.108	117.8 ± 0.8
weighted avg	0.786 ± 0.115	0.788 ± 0.071	0.765 ± 0.089	117.8 ± 0.8

Configuration:

```
{
  semi_supervised: {
    mean_teacher_config: {
      ema_decay: 0.99
    },
    semi_supervised_config: {
      model_name: mean_teacher,
      feature_extractor_name: resnet18,
      classifier_name: linear,
      training: {
        consistency_lambda_max: 0.1,
        consistency_loss_type: cross_entropy,
        ramp_up_epochs: 100,
        ramp_up_function: linear
      }
    }
  },
  architectures: {
    classifiers_config: {
      linear: {},
    },
    feature_extractors_config: {
```

```

        resnet18: {
            output_size: 64,
            pretrained: True,
            freeze_feature_extractor: True,
            dropout_rate: 0.2,
        },
    }
},
general_config: {
    data: {
        unlabeled_sample_size: 2000,
        labeled_sample_size: 1000000000.0,
        preload_data: True,
        apply_transforms_on_the_fly: False,
        normalize_data: True,
        normalize_mean: [0.485, 0.456, 0.406],
        normalize_std: [0.229, 0.224, 0.225],
        use_extra_features: True,
        extra_feature_dim: 2,
        number_of_augmentations: 3
    },
    training: {
        test_size: 0.2,
        val_size: 0.1,
        batch_size: 64,
        num_epochs: 500,
        optimizer: adamw,
        learning_rate: 0.0001,
        weight_decay: 1e-05,
        early_stop: 2,
        early_stop_check_val_every_n_epoch: 5,
        cv_folds: 5
    },
    system: {
        random_seed: 44
    },
},
}

```

6- Self-supervised models:

The self-supervised runner contains two main steps: 1- pretraining the feature extractor solely using unsupervised samples, 2- fine-tuning the feature extractor and train a classifier on top of that. Just like the supervised and semi-supervised models the self-supervised models rely on predefined and pretrained feature extractors like resnet18. The model is further pretrained on the WSI data with the goal of minimizing some consistency loss. Further on the pretrained feature extractor is used in the self supervised manner with linear probing method to learn the classification of the labeled data samples.

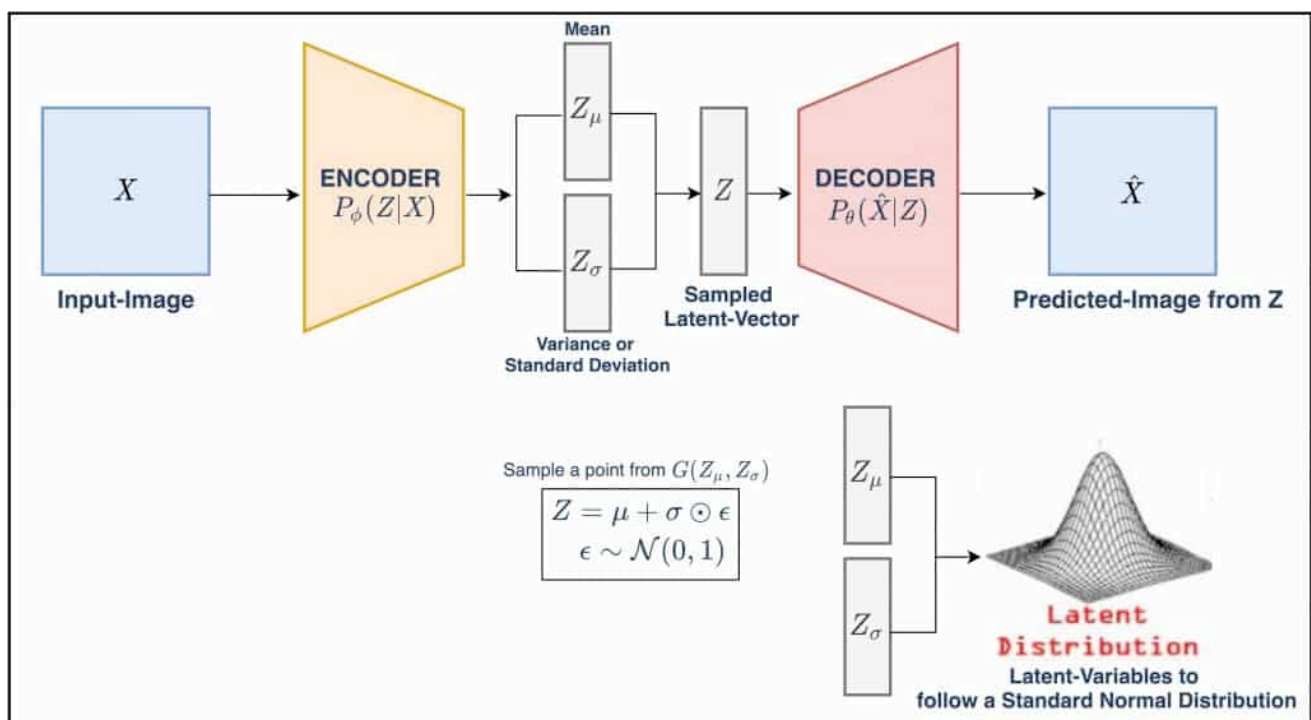
6-1 VAE:

A **Variational Autoencoder (VAE)** is a **generative latent-variable model** that learns a compact and continuous representation of data in an **unsupervised** manner. Unlike classical autoencoders, which deterministically map inputs to latent codes, a VAE models the latent representation **probabilistically**, enabling both meaningful representation learning and data generation.

The core assumption of a VAE is that each observed data x is generated from an unobserved latent z , which follows a known prior distribution, typically a standard normal distribution $p(z)=N(0,I)$

The goal of the model is to learn how to encode data into this latent space and decode latent variables back into the original data space.

For this model I used a predefined feature extractor (resnet18) for the encoder and for the decoder defined a simple sequence of ConvTranspose2D layers.



Results of 5-fold cross validation:

	precision	recall	f1-score	support
CAA	0.882 ± 0.1	0.877 ± 0.194	0.874 ± 0.09	27.0 ± 0.0
Coarse	0.883 ± 0.101	0.887 ± 0.08	0.884 ± 0.068	35.667 ± 0.943
Compact	0.904 ± 0.157	0.937 ± 0.09	0.916 ± 0.048	21.0 ± 0.0
Cored	0.851 ± 0.018	0.873 ± 0.098	0.862 ± 0.057	26.333 ± 0.943
Diffuse	0.92 ± 0.059	0.906 ± 0.036	0.912 ± 0.014	24.667 ± 0.943
NonAB	0.842 ± 0.057	0.931 ± 0.104	0.883 ± 0.015	24.333 ± 0.943
OtherAB	0.491 ± 0.224	0.479 ± 0.276	0.484 ± 0.247	10.333 ± 0.943
Subpial	0.759 ± 0.253	0.675 ± 0.059	0.712 ± 0.146	14.333 ± 0.943
UndefAB	0.891 ± 0.209	0.712 ± 0.38	0.784 ± 0.306	12.667 ± 0.943
macro avg	0.825 ± 0.063	0.808 ± 0.053	0.812 ± 0.062	196.333 ± 0.943
weighted avg	0.851 ± 0.045	0.849 ± 0.043	0.846 ± 0.047	196.333 ± 0.943

Configuration:

```
{
  architectures: {
    classifiers_config: {
      linear: {

      },
    },
    feature_extractors_config: {
      resnet18: {
        output_size: 64,
        pretrained: True,
        freeze: False,
        unfreeze_after_n_epochs: 0,
        unfreeze_last_n_blocks: 0,
        dropout_rate: 0.2,
      },
    },
  },
  self_supervised: {
    vae_config: {
      latent_dim: 64,
      beta: 1.0,
      reconstruction_loss: mse
    },
    self_supervised_config: {
      pretraining_method: vae,
      feature_extractor_name: resnet18,
      classifier_name: linear,
      pretraining: {
        skip_if_checkpoint_exists: True,
        checkpoint_path: checkpoints,
        num_epochs: 50,
        learning_rate: 0.0001,
        weight_decay: 1e-05
      }
    },
  },
  general_config: {
    data: {
      unlabeled_sample_size: 2000,
      labeled_sample_size: 1000000000.0,
      preload_data: True,
      apply_transforms_on_the_fly: False,
      normalize_data: True,
      normalize_mean: [0.485, 0.456, 0.406],
      normalize_std: [0.229, 0.224, 0.225],
    },
  },
}
```

```
        use_extra_features: True,
        extra_feature_dim: 2,
        number_of_augmentations: 3
    },
    training: {
        test_size: 0.2,
        val_size: 0.1,
        batch_size: 64,
        num_epochs: 500,
        optimizer: adamw,
        learning_rate: 1e-05,
        weight_decay: 1e-05,
        early_stop: 2,
        early_stop_check_val_every_n_epoch: 5,
        cv_folds: 3,
        use_thresholding: True,
        threshold_min: 0,
        threshold_max: 1,
        threshold_steps: 20
    },
},
}
```

6-1 SimCLR:

SimCLR (Simple Framework for Contrastive Learning of Visual Representations) is a self-supervised learning method that learns useful image features without needing labels by using **contrastive learning**. The core idea is to take an image and generate two different augmented views of it (for example through cropping, color jittering, or blurring), then train a neural network to produce similar embeddings for these two views while pushing apart embeddings from different images in the same batch. This is done using a contrastive loss (NT-Xent loss) and a projection head on top of the encoder. After training, the projection head is discarded and the encoder can be used as a strong feature extractor for downstream tasks like classification, clustering, or segmentation, often performing competitively with supervised pretraining.

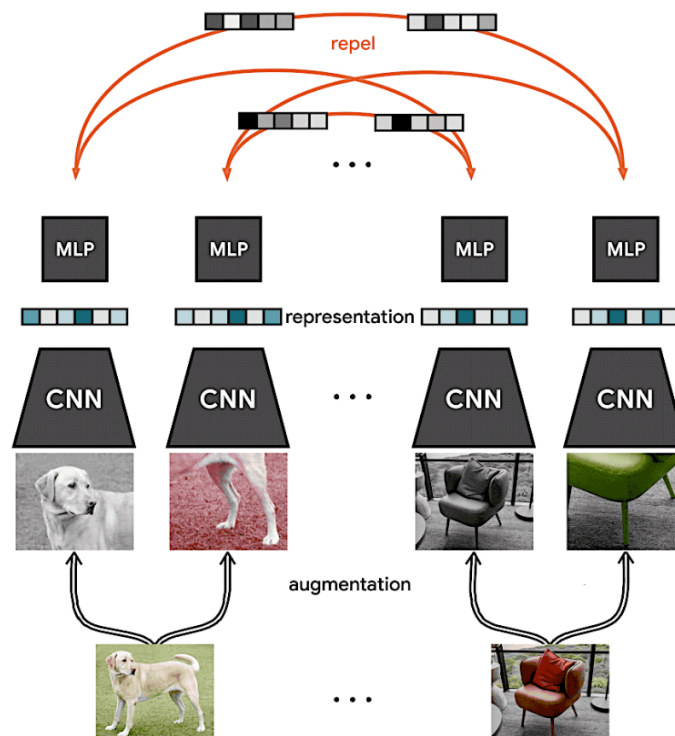
$$\ell_{i,j} = -\log \frac{\exp\left(\frac{\text{sim}(z_i, z_j)}{\tau}\right)}{\sum_{k=1}^{2N} \mathbf{1}_{[k \neq i]} \exp\left(\frac{\text{sim}(z_i, z_k)}{\tau}\right)}$$

Where:

- z_i, z_j are the projected embeddings of two **positive pairs** (two augmented views of the same image).
- $2N$ is the total number of augmented samples in a batch (since each of the N images has 2 views).
- $\text{sim}(z_i, z_j)$ is the cosine similarity:

$$\text{sim}(z_i, z_j) = \frac{z_i^\top z_j}{\|z_i\| \|z_j\|}$$

- τ is the **temperature** hyperparameter controlling the sharpness of the distribution.
- $\mathbf{1}_{[k \neq i]}$ excludes the anchor sample itself from the denominator.



Results of 5-fold cross validation:

classification_report

	precision	recall	f1-score	support
CAA	0.874 ± 0.133	0.963 ± 0.043	0.914 ± 0.075	20.25 ± 0.866
Coarse	0.924 ± 0.049	0.897 ± 0.033	0.91 ± 0.029	26.75 ± 0.866
Compact	0.918 ± 0.177	0.92 ± 0.061	0.916 ± 0.08	15.75 ± 0.866
Cored	0.855 ± 0.159	0.885 ± 0.184	0.864 ± 0.105	19.75 ± 0.866
Diffuse	0.905 ± 0.124	0.959 ± 0.048	0.93 ± 0.05	18.5 ± 1.0
NonAB	0.881 ± 0.208	0.946 ± 0.108	0.911 ± 0.16	18.25 ± 0.866
OtherAB	0.676 ± 0.499	0.424 ± 0.245	0.504 ± 0.257	7.75 ± 0.866
Subpial	0.742 ± 0.25	0.606 ± 0.131	0.66 ± 0.118	10.75 ± 0.866
UndefAB	0.795 ± 0.106	0.692 ± 0.363	0.724 ± 0.229	9.5 ± 1.0
macro avg	0.841 ± 0.081	0.81 ± 0.083	0.815 ± 0.083	147.25 ± 0.866
weighted avg	0.864 ± 0.062	0.861 ± 0.065	0.856 ± 0.066	147.25 ± 0.866

Configuration:

```
{
  architectures: {
    classifiers_config: {
      linear: {

    },
  },
  feature_extractors_config: {
    resnet18: {
      output_size: 64,
      pretrained: True,
      freeze: False,
      unfreeze_after_n_epochs: 0,
      unfreeze_last_n_blocks: 0,
      dropout_rate: 0.2,
    },
  },
},
self_supervised: {
  simclr_config: {
    temperature: 0.2,
    projection_head_sizes: [64],
    projection_head_activations: relu
  },
  self_supervised_config: {
    pretraining_method: similar,
    feature_extractor_name: resnet18,
    classifier_name: linear,
    pretraining: {
      skip_if_checkpoint_exists: True,
      num_epochs: 50,
      learning_rate: 1e-05,
      weight_decay: 1e-05
    }
  },
},
general_config: {
  data: {
    unlabeled_sample_size: 5000,
    labeled_sample_size: 1000000000.0,
    preload_data: True,
    apply_transforms_on_the_fly: False,
    normalize_data: True,
```

```
        normalize_mean: [0.485, 0.456, 0.406],
        normalize_std: [0.229, 0.224, 0.225],
        use_extra_features: True,
        extra_feature_dim: 2,
        number_of_augmentations: 3
    },
    training: {
        test_size: 0.25,
        val_size: 0.1,
        batch_size: 64,
        num_epochs: 500,
        optimizer: adamw,
        learning_rate: 1e-05,
        weight_decay: 1e-05,
        early_stop: 2,
        early_stop_check_val_every_n_epoch: 5,
        use_thresholding: True,
        threshold_min: 0,
        threshold_max: 1,
        threshold_steps: 5
    },
},
}
```


To be Implemented:

1- Improve the current Semi-supervised models: -> Deadline: 15 December

Following you can find some possible ways I plan to improve the models:

1- One challenge the semi-supervised models currently face is premature early stopping, caused by the consistency loss introducing noise into the validation loss. My goal is to explore potential improvements to address this issue.

Skipped this point since it may not be the main focus of the project.

2- Additionally, the reported test metrics show high variance sometimes up to 20% which is not ideal. One possible solution is to use an ensemble of models for each component, though this would increase training time linearly.

It seems like the the variance I'm seeing is mostly a **data issue**, not a bug in the code: per-fold test sets are very small per class, so a handful of images dominate each metric, especially for the rare classes. When using cross validation with 5-folds there are some classes like OtherAB, Subpial and UndefAB which have only about 6 samples in the test set and considering that a single misclassification changes the recall by about $1/6 \approx 0.17$. Upon testing the cross validation with fewer folds I realized the performance metrics have much less variance (half of the original variance) since the size of the test data doubled from when I performed 5-fold cross validation.

3- Another aspect worth investigating is the class imbalance problem. While the current implementation uses stratified splits to preserve class distributions across validation and test sets, the classification still relies on the argmax method. A potential improvement is to determine class-specific classification thresholds using the validation set, which may help mitigate the effects of imbalanced classes. This is the approach used in the original thesis paper and I believe it adds a lot to the performance and is easy to implement.

Added classification thresholds and it looks like the results improved a bit. Its interesting to note that the classes ended up having so different thresholds based on the validation set scores. Example:

[Diffuse: 0.6, Cored: 0.2, Compact: 0.05, Coarse: 0.2, CAA: 0.1, Subpial: 0.35, OtherAB: 0.1, UndefAB: 0.25, NonAB: 0.1]

4- So far, the feature extractors have been frozen during training. One possible direction is to unfreeze the last few layers of the ResNet and fine-tune them.

I also implemented this but it turned out that fine tuning the whole feature extractor backbone achieves a better performance than only unfreezing the last few blocks.

2- Self-supervised models: -> Deadline: 31 January

The final phase of this project involves exploring **self-supervised learning** methods. I plan to investigate two complementary directions:

1. **Trainable self-supervised models** such as autoencoders and SimCLR, where the goal is to first learn meaningful image representations and then train a classifier on top. In this approach, we explicitly train a feature extractor in a self-supervised manner and then attach a classification head for downstream plaque classification.

The models are implemented and evaluated using cross validation. It looks like overall self supervised models perform the best out of all the models with on average 4 percent improvement on the overall f1-score in compare to the Supervised module and previous paper results.

2. **Pretrained state-of-the-art biomedical self-supervised models**, which can serve directly as feature extractors. Here, the focus will be on evaluating existing SSL models developed for histopathology or biomedical imaging, followed by training a lightweight classification head on the extracted features. One of the state of the art models in this area is **H-optimus** which is a 1.1B parameter vision transformer trained with self-supervised learning on an extensive proprietary dataset of billions of histology images sampled from over 1 million slides of more than 800,000 patients. I believe extracting features shouldn't be challenging considering the nice documentation they have on [huggingface](#).

I have submitted my request to access the repository on hugging face and its been a week that I have been waiting for access from authors.

Hugging Face Search models, datasets, users...

Models Datasets Spaces Community Docs Enterprise Pricing

bioptimus/H-optimus-1 like 34 Follow Bioptimus 87

Image Feature Extraction timm PyTorch Safetensors pathology histology medical imaging self-supervised learning vision transformer foundation model License: cc-by-nc-nd-4.0

Model card Files and versions xet Community 18 Use this model

You need to agree to share your contact information to access this model

This repository is publicly accessible, but you have to accept the conditions to access its files and content.

- This model and associated code are released under the CC-BY-NC-ND 4.0 license and may only be used for non-commercial, academic research purposes with proper attribution.
- Any commercial use, sale, or other monetization of the H-optimus-1 model and its derivatives, which include models trained on outputs from the H-optimus-1 model or datasets created from the H-optimus-1 model, is prohibited and requires prior approval.
- Please note that the primary email used to sign up for your Hugging Face account must match your institutional email to receive approval. By downloading the model, you attest that all information (affiliation, research use) is correct and up-to-date. Download...

Your request to access this repository has been submitted and is awaiting a review from the repository authors. You can check the status of all your access requests in [your settings](#).

Downloads last month: **4,747**

Inference Providers NEW

Image Feature Extraction

This model isn't deployed by any Inference Provider. [Ask for provider support](#)

Collection including bioptimus/H-optimus-1

H-models Collection

2 items • Updated Aug 20, 2025

3- Hyperparameter optimization: -> Deadline: 11 March

Hyperparameter selection plays a crucial role in training our models. It spans both general hyperparameters such as learning rate and model-specific ones, including dropout rates for ResNet feature extractors or confidence thresholds in semi-supervised methods like FixMatch. To systematically search for an optimal configuration, I plan to employ **Optuna**, which integrates seamlessly with the PyTorch Lightning framework and supports efficient hyperparameter optimization.

However, this process presents notable challenges. The primary bottleneck is runtime: for context, a single supervised training run with 200 epochs and early stopping requires approximately **10 minutes** on the DAIC cluster. Extending this to a 5-fold cross-validation setup multiplies the runtime to **$N \times 50$ minutes**, where N is the number of hyperparameter configurations being evaluated. Given the compute limitations and queue times on the cluster, running full cross-validation for every trial is likely infeasible.

The hyperparameter tuning pipeline is implemented using Optuna. In this phase, only a subset of the most important hyperparameters is tuned, while the remaining ones are kept fixed, since varying them is unlikely to significantly affect the model's performance.

Hyperparameter configurations are sampled using the Tree-structured Parzen Estimator (TPE) heuristic provided by Optuna. This approach allows the search procedure to explore promising regions of the parameter space without exhaustively performing a grid search over the entire space. As a result, near-optimal hyperparameter configurations can be identified more efficiently.

To reduce computational complexity, nested cross-validation is not used during the hyperparameter tuning phase. Instead, no explicit test set is defined at this stage. The entire dataset is used within a k-fold cross-validation framework where validation folds are rotated. For each hyperparameter configuration, the model is trained and evaluated across the validation folds, and the average validation loss is used to determine the best set of hyperparameters for each model.

Once the optimal hyperparameters are identified, a separate evaluation phase will be performed. In this phase, cross-validation will be executed again using a clear train–test split, and the final performance of the model will be reported based on the test sets. This step does not require additional implementation, as the cross-validation pipeline for models with fixed hyperparameters has already been developed.

In summary, the adopted procedure is as follows:

- During hyperparameter tuning, the entire dataset is used.
- K-fold cross-validation is applied with rotating validation folds.
- No dedicated test set is used during this stage.
- After selecting the best hyperparameters, a separate cross-validation run with fixed hyperparameters is performed to evaluate the model and report its performance on the test sets.

So far, hyperparameter tuning has been successfully tested for the supervised module. However, some constraints were encountered that limited further experimentation.

trial_number	mean_cv_loss	std_cv_loss	mean_cv_accuracy	std_cv_accuracy	fe_resnet18.dropout_rate	fe_resnet18.freeze	fe_resnet18.output_size	fe_resnet50.dropout_rate	fe_resnet50.freeze
0	0.372	0.060	90.152	2.051				0.1	TRUE
1	0.429	0.036	89.471	1.761	0.2	TRUE	128		

fe_resnet50.output_size	general_config.training.batch_size	general_config.training.learning_rate	general_config.training.weight_decay	supervised.supervised_config.classifier_name	supervised.supervised_config.dropout_rate
128	32	1E-05	1E-05	linear	resnet50
	32	5E-05	1E-06	linear	resnet18

Apparently with the current access I have to the HPC server I'm not allowed to run jobs for more than 4 hours due to the QOS policy. If I get this sorted out then I will be able to run the hyperparmater tuning for all the other modules and I can select the best hyper parameters for each model and then run a final cross validation on the model with the best hyperparams and report the performance of each model in my papers Evaluation section.

```
batch_scripts > $ main.sbatch
```

```
1  #!/bin/sh
2
3  #SBATCH --partition=general
4  #SBATCH --qos=short
5  #SBATCH --time=12:00:00
6  #SBATCH --ntasks=1
7  #SBATCH --cpus-per-task=4
8  #SBATCH --mem=10G
9  #SBATCH --mail-type=END
10 #SBATCH --gres=gpu
11 #SBATCH --output=logs/%x-%j.out
12 #SBATCH --error=logs/%x-%j.err
13
14 module use /opt/insy/modulefiles
15 module load cuda/10.0 cudnn/10.0-7.4.2.24
16
17 # Ensure typing_extensions has 'deprecated' (required by PyTorch on Python 3.8)
18 python -m pip install "typing_extensions>=4.5.0" --quiet --user
19
20 srun python src/main.py --config_dir configs --train_mode supervised --run_mode hyperparameter_tuning --n_trials 2
21
22
```

Problems Output Debug Console Terminal Ports

```
(pytorch) bash-4.2$ sbatch batch_scripts/main.sbatch
sbatch: error: QOSMaxWallDurationPerJobLimit
sbatch: error: Batch job submission failed: Job violates accounting/QOS policy (job submit limit, user's size and/or time limits)
(pytorch) bash-4.2$
```

3- Hyperparameter optimization (Running the experiments):

-> Deadline 1 April

After resolving the issue with the job submission time limit on the server, I expect that running the hyperparameter tuning experiments for all three modules (Supervised, Semi-supervised, and Self-supervised) should take less than a week. By the end of this period, the goal is to produce a table comparing the performance of the different models along with their corresponding best hyperparameters.

4- Report Writing

-> Deadline 1 June

A more fine grained schedule for the report writing will be discussed in the next meeting.

Objective performance scores to beat from previous study

Table 3.2: A summary of the average performance of the trained models across all considered A β classes is given in percentages. The performance metrics consist of accuracy, precision, recall, F1 score, and Receiver Operating Characteristic Area Under the Curve (ROC AUC). Each metric is reported with its mean value $\pm$ standard deviation computed across five cross-validation configurations, on the corresponding test fold for each model. The overall metrics are computed based on all nine classes.

Class	Accuracy	Precision	Recall	F1 Score	ROC AUC
Overall	76.83 $\pm$ 1.81	82.49 $\pm$ 2.39	76.83 $\pm$ 1.81	77.00 $\pm$ 1.51	95.84 $\pm$ 0.40
Overall (Primary)	85.71 $\pm$ 2.38	89.47 $\pm$ 3.24	85.71 $\pm$ 2.38	86.61 $\pm$ 0.71	97.30 $\pm$ 0.75
Overall (Anomaly)	59.05 $\pm$ 2.61	68.54 $\pm$ 9.80	59.05 $\pm$ 2.61	57.79 $\pm$ 5.23	92.91 $\pm$ 1.32
Diffuse	-	92.14 $\pm$ 11.41	82.86 $\pm$ 6.39	86.73 $\pm$ 5.48	97.81 $\pm$ 1.52
Cored	-	100.00 $\pm$ 0.00	68.57 $\pm$ 6.39	81.21 $\pm$ 4.74	89.64 $\pm$ 2.72
Compact	-	92.14 $\pm$ 7.21	91.43 $\pm$ 12.78	91.14 $\pm$ 6.68	99.80 $\pm$ 0.28
Coarse	-	74.00 $\pm$ 2.24	88.57 $\pm$ 6.39	80.47 $\pm$ 1.05	97.45 $\pm$ 2.06
CAA	-	97.50 $\pm$ 5.59	100.00 $\pm$ 0.00	98.67 $\pm$ 2.98	100.00 $\pm$ 0.00
Subpial	-	81.03 $\pm$ 4.46	82.86 $\pm$ 11.95	81.41 $\pm$ 4.95	99.13 $\pm$ 0.29
OtherAB	-	74.22 $\pm$ 20.46	57.14 $\pm$ 0.00	63.52 $\pm$ 8.53	93.32 $\pm$ 2.54
UndefAB	-	83.33 $\pm$ 23.57	40.00 $\pm$ 11.95	51.56 $\pm$ 9.08	94.23 $\pm$ 1.03
NonAB	-	48.06 $\pm$ 11.51	80.00 $\pm$ 12.78	58.28 $\pm$ 4.71	91.17 $\pm$ 4.02